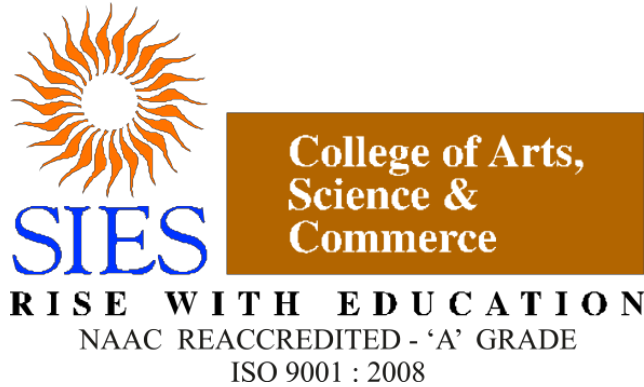




S.I.E.S College of Arts, Science and Commerce (Empowered Autonomous)
Sion(W), Mumbai – 400 022.

CERTIFICATE

This is to certify that Miss/Mr Vimlesh Mourya Roll No.FMCS2526111 has successfully completed the necessary course of experiments in the subject of **Internet Of Things** during the academic year **2025 – 2026** complying with the requirements of **University of Mumbai**, for the course of MSCCS [**Semester- II**].

Asst. Prof. In-Charge

Examination date:

Examiner's Signature & Date:

College Seal

Head of Department
DR. MANOJ SINGH

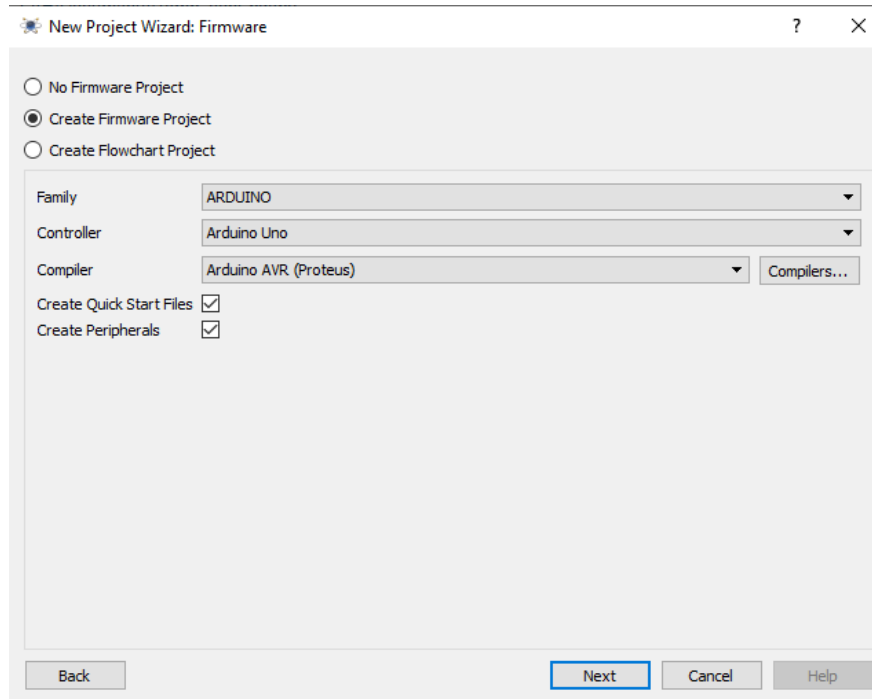
INDEX

Sr. no.	Practical's	Date	Sign
1	Design a temperature controller using a microcontroller, use LM35 as temperature sensor and relay to control the heater (PR)	16/01/2026	
2	Design a Embedded System to control Lighting using LDR as sensor use Relay to control the Lightings (PR)	20/01/2026	
3	Design and demonstrate communication between 2 embedded devices (microcontroller) (PR)	23/01/2026	
4	Design a Visitor Counter using a microcontroller to count the number of visitors entering a hall . Use sensor and LCD to display (CPK)	27/01/2026	
5	Develop iot application to detect carbon monoxide gas. If level if high then must open window/door and activate high speed fan (CPK)	03/02/2026	
6	Develop iot application for motion detection sensor. When motion is detected siren and web cam, and door is activated and deactivated when not present (CPK)	05/02/2026	
7	Develop an IOT application to control sprinklers in the field using sensor. (CPK)	10/02/2026	
8	Develop an IOT application for monitoring water levels in tanks and automatically start the motor to fill the tank if the level goes below the critical level. (CPK)	04/03/2026	
9	Motion Garage if motion is detected the garage door should open and should be controlled using a smart phone.	19/03/2026	

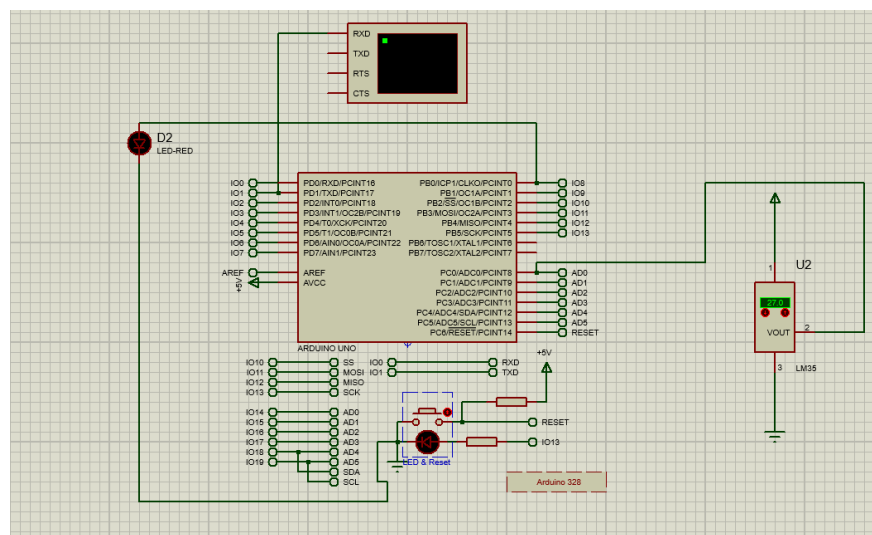
PRACTICAL NO: 01

Aim: Design a temperature controller using a microcontroller, use LM35 as temperature sensor and relay to control the heater.

Step 1 : Open Proteus as administrator, create a new project, then next next next, then select create firmware and select arduino , then next and finish.



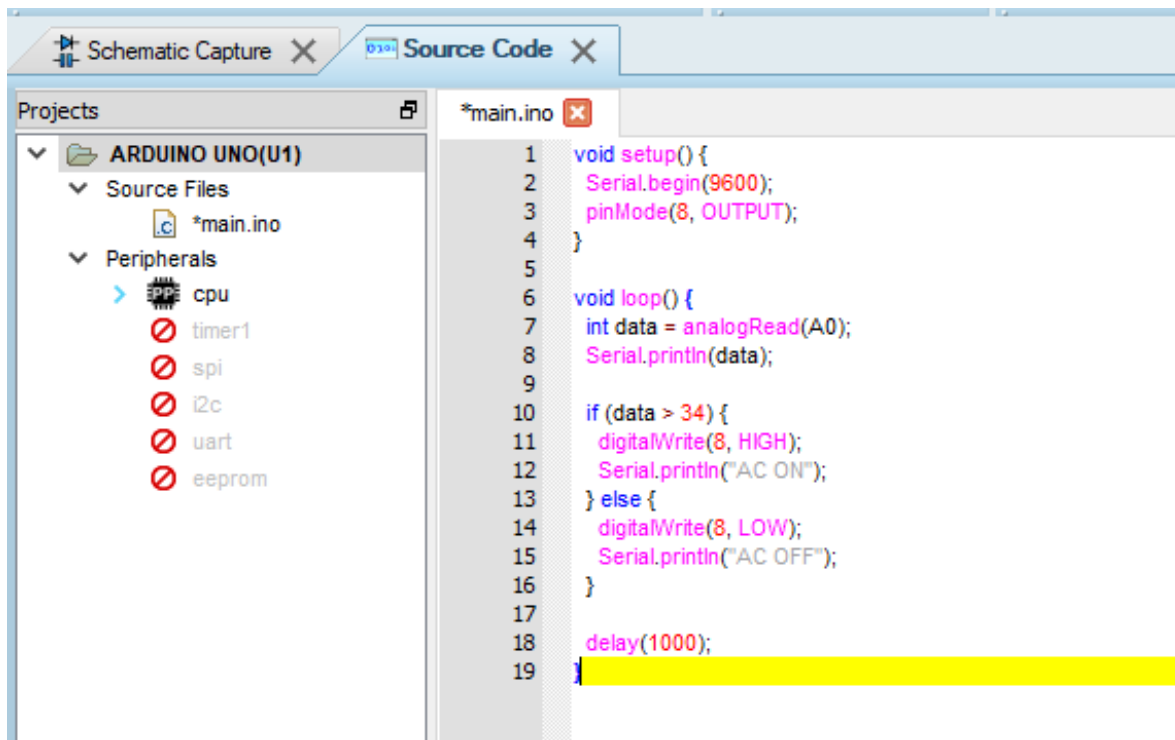
Step 2 : Create circuit design with the right components.

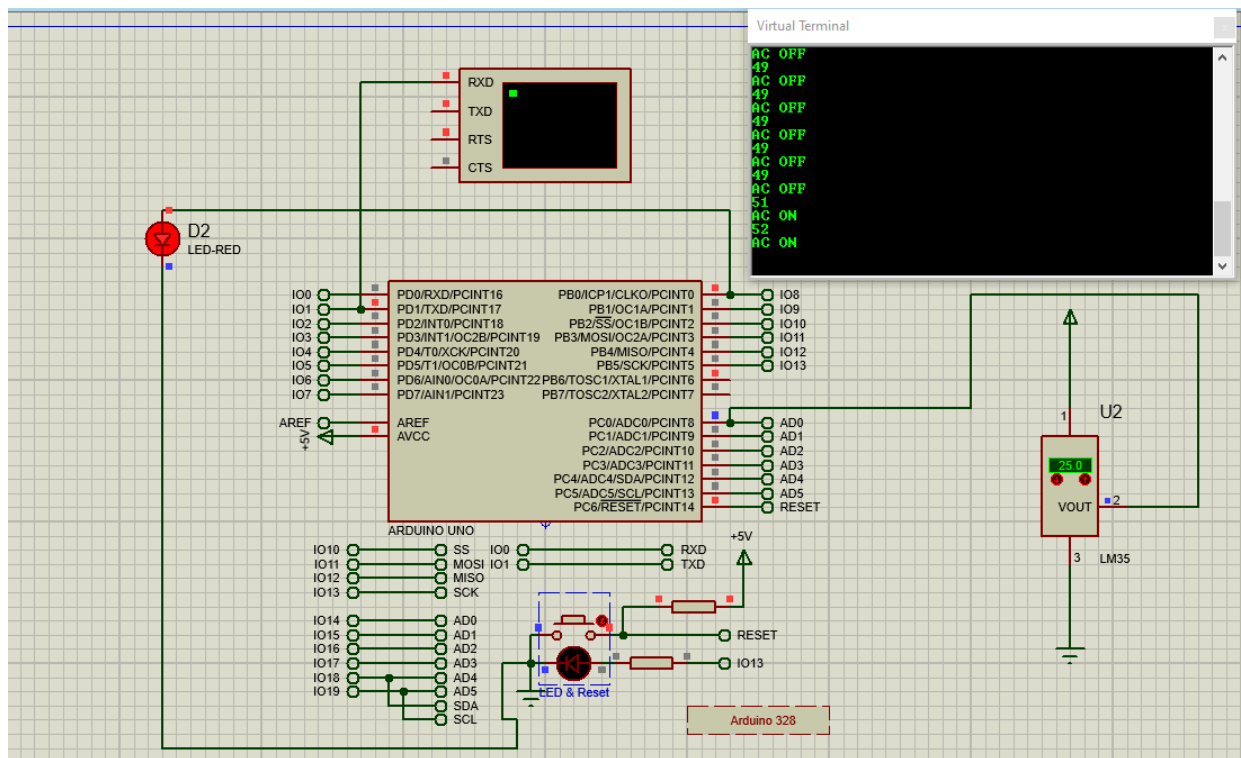
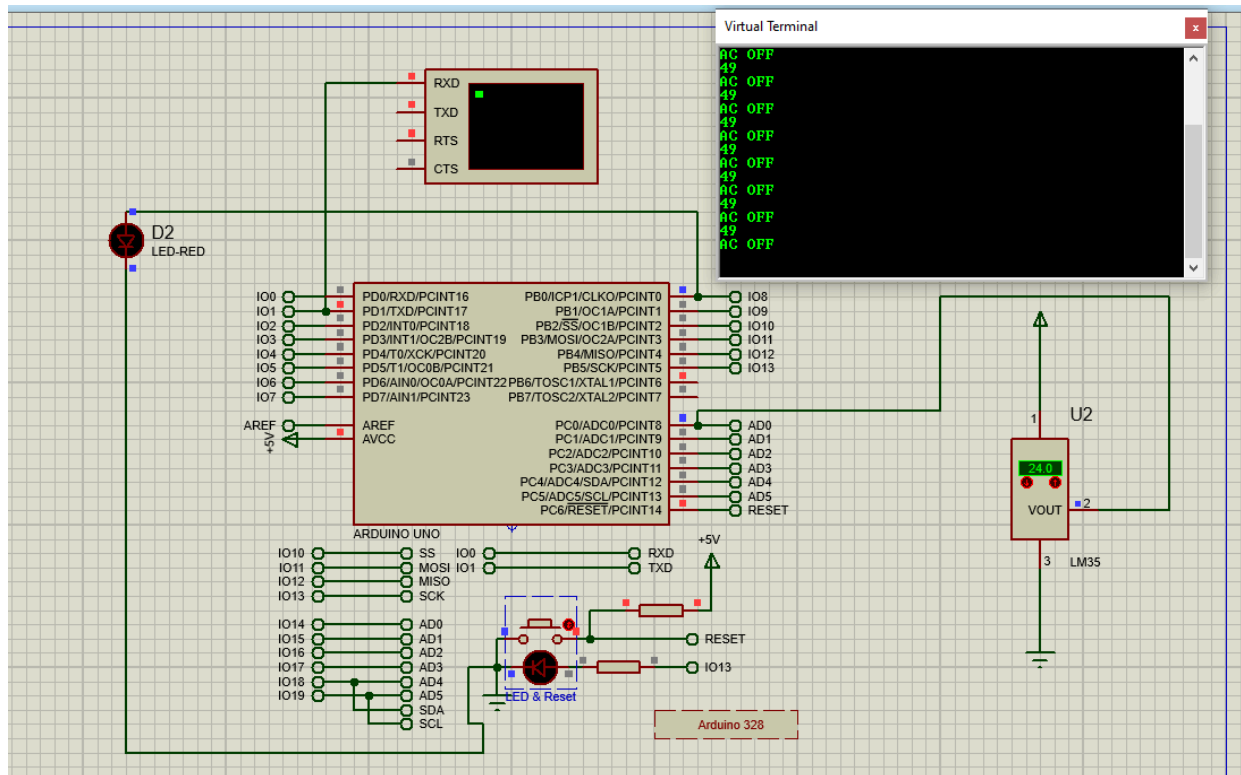


Step 3 : Write the code for the circuit.

Code :

```
void setup() {  
  Serial.begin(9600);  
  pinMode(8, OUTPUT);  
}  
  
void loop() {  
  int data = analogRead(A0);  
  Serial.println(data);  
  
  if (data > 50) {  
    digitalWrite(8, HIGH);  
    Serial.println("AC ON");  
  } else {  
    digitalWrite(8, LOW);  
    Serial.println("AC OFF");  
  }  
  
  delay(1000);  
}
```

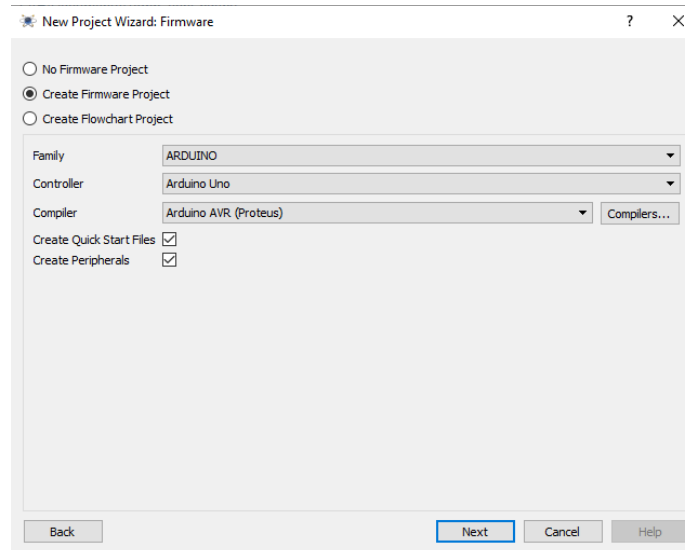


Output :

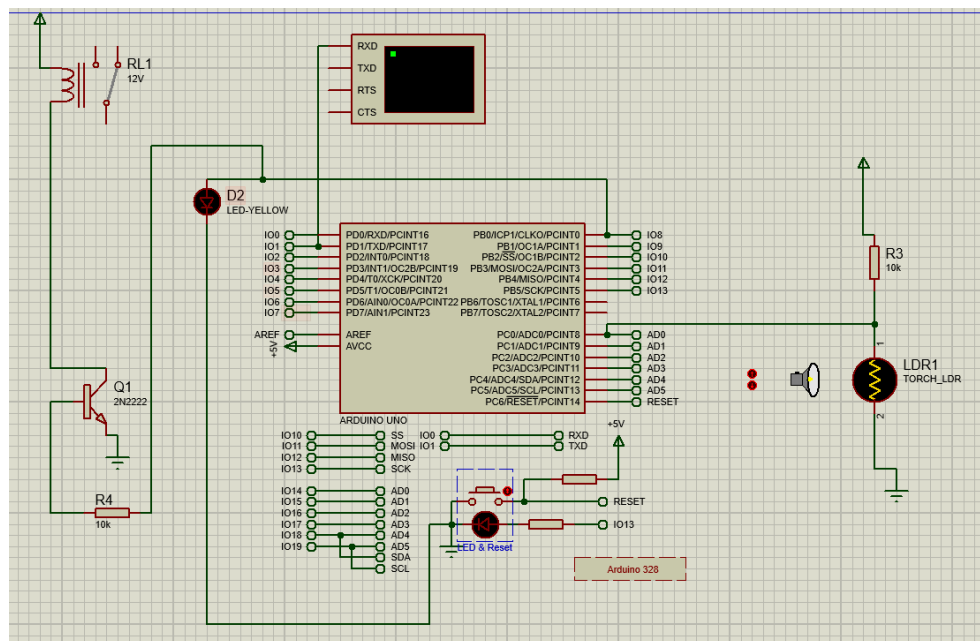
PRACTICAL NO: 02

Aim: Design an Embedded System to control Lighting using LDR as sensor use Relay to control the Lightings (PR)

Step 1 : Open Proteus as administrator, create a new project, then next next next, then select create firmware and select arduino , then next and finish.



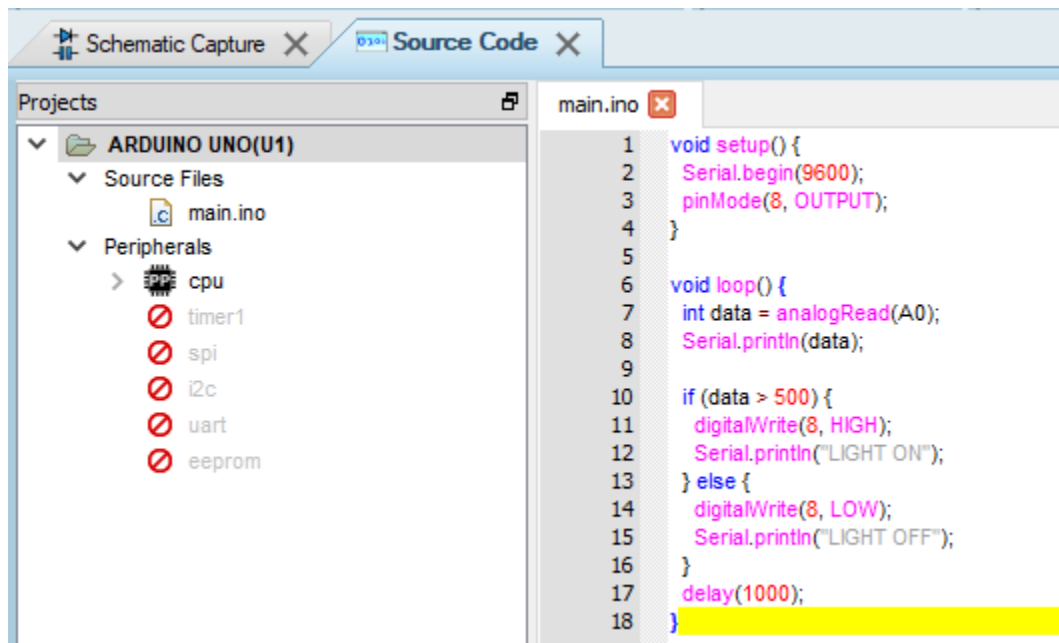
Step 2 : Create circuit design with the right components.

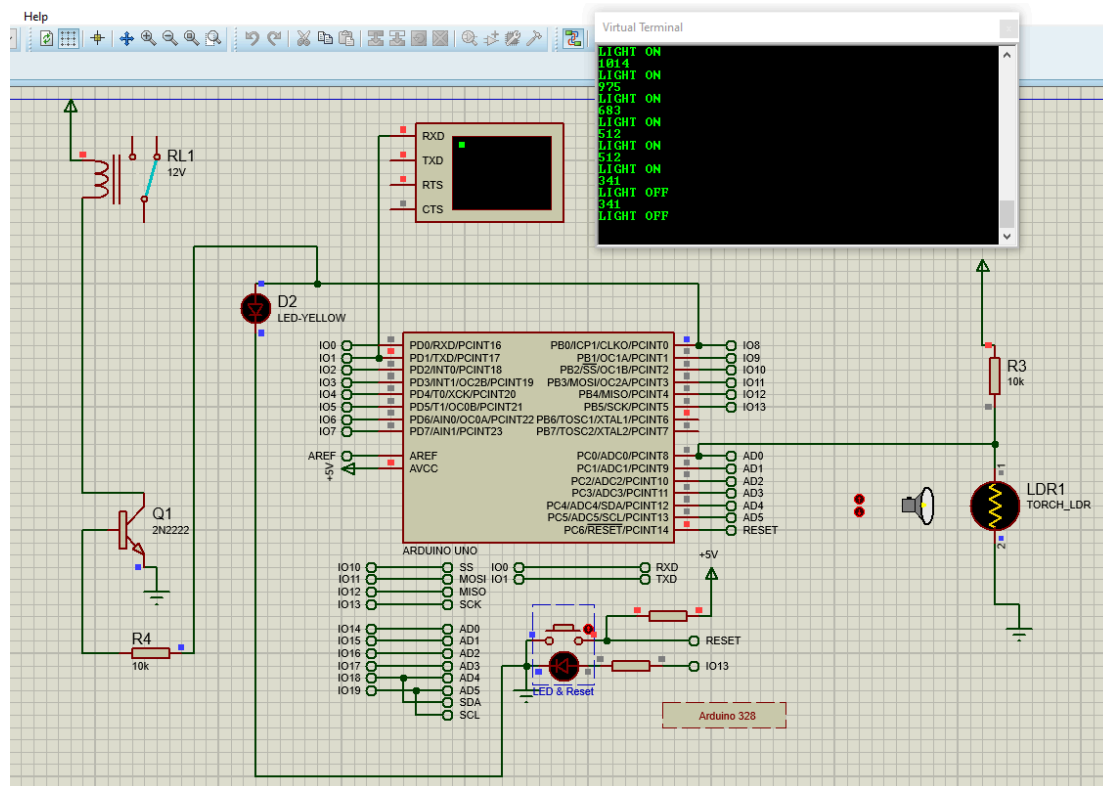
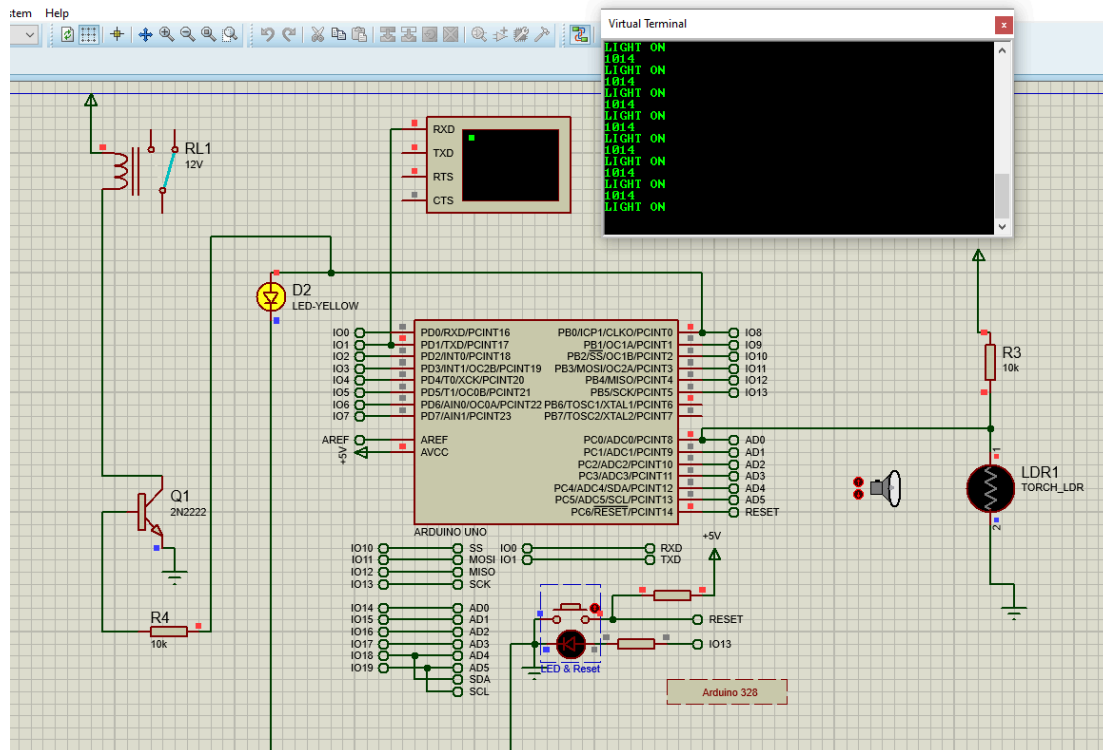


Step 3 : Write the code for the circuit.

Code :

```
void setup() {  
  Serial.begin(9600);  
  pinMode(8, OUTPUT);  
}  
  
void loop() {  
  int data = analogRead(A0);  
  Serial.println(data);  
  
  if (data > 500) {  
    digitalWrite(8, HIGH);  
    Serial.println("LIGHT ON");  
  } else {  
    digitalWrite(8, LOW);  
    Serial.println("LIGHT OFF");  
  }  
  delay(1000);  
}
```

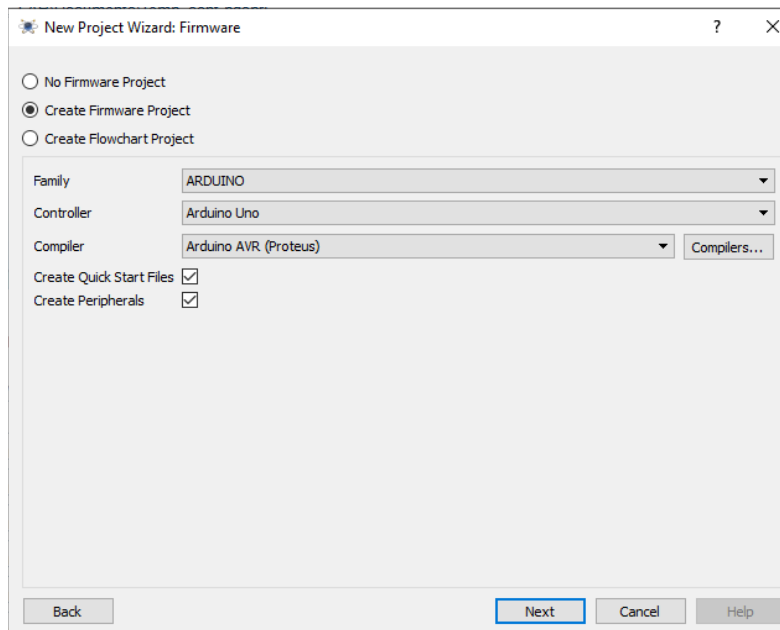


Output :

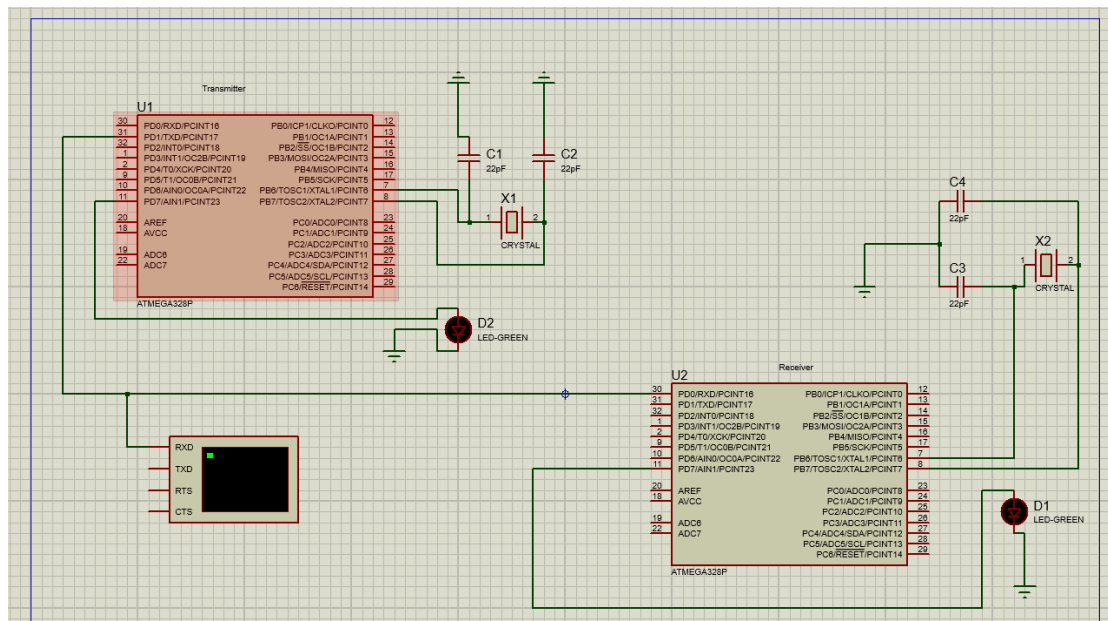
PRACTICAL NO: 03

Aim: Design and demonstrate communication between 2 embedded devices (microcontroller) (PR)

Step 1 : Open Proteus as administrator, create a new project, then next next next, then select create firmware and select arduino , then next and finish.



Step 2 : Create circuit design with the right components.



Step 3 : Write the code for the circuit.

Code :

Transmitter

Receive

```
sketch_mar20a
void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  pinMode(7,OUTPUT);
}

int thisByte = 85;

void loop() {
  // put your main code here, to run repeatedly:
  delay(1000);
  digitalWrite(7,1);
  Serial.write(thisByte);
}
```

```
sketch_mar20b
void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  pinMode(7,OUTPUT);
}

void loop() {
  // put your main code here, to run repeatedly:
  byte brightness;
  if(Serial.available()){
    brightness = Serial.read();
    digitalWrite(7,1);
  }
}
```

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  pinMode(7,OUTPUT);
}

int thisByte = 85;

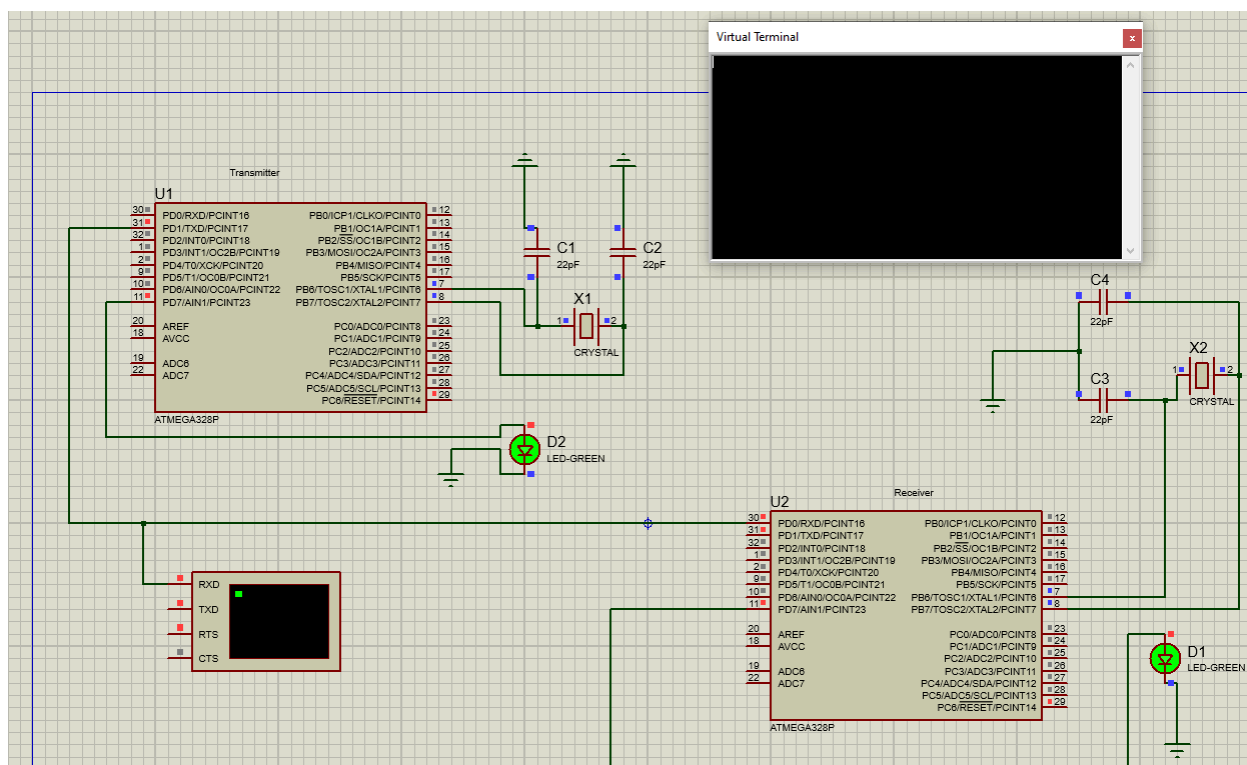
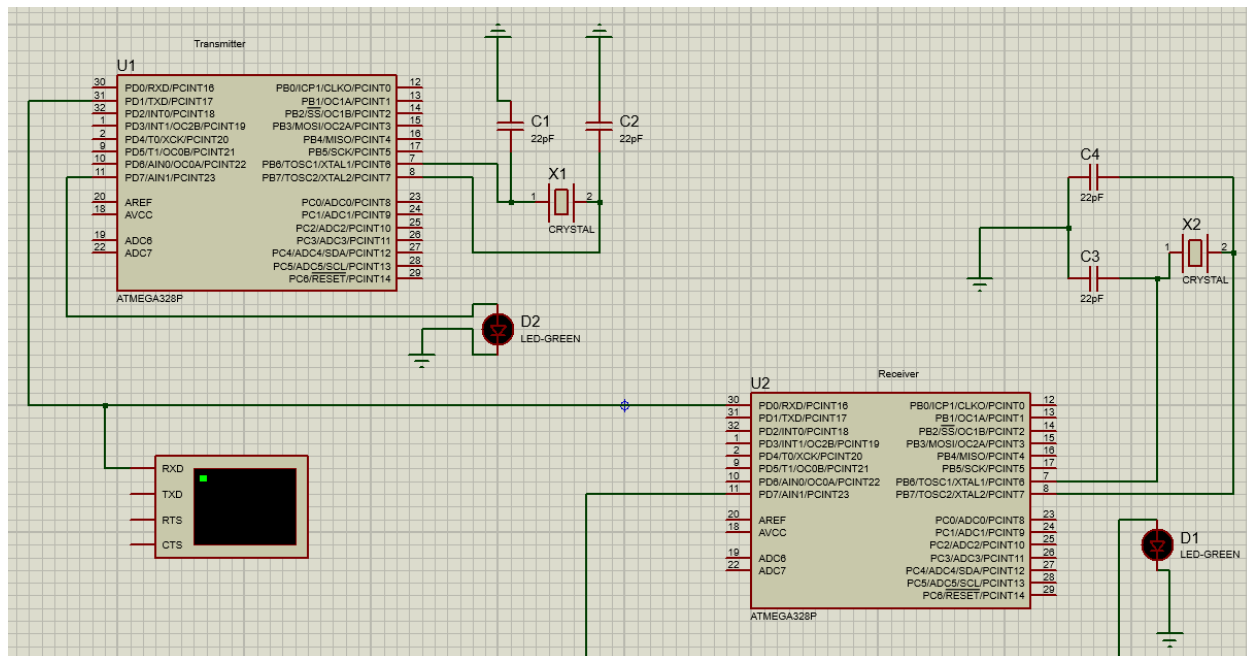
void loop() {
  // put your main code here, to run repeatedly:
  delay(1000);
  digitalWrite(7,1);
  Serial.write(thisByte);
}
```

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  pinMode(7,OUTPUT);
}

void loop() {
  // put your main code here, to run repeatedly:
  byte brightness;
  if(Serial.available()){
    brightness = Serial.read();
  }
}
```

```
digitalWrite(7,1);
```

Output :



PRACTICAL NO: 04

Aim: To design and implement a visitor counting system using a motion sensor in Cisco Packet Tracer that displays the number of visitors on an LCD.

Description

This project demonstrates an IoT-based visitor counting system using a **motion sensor** connected to a microcontroller (MCU).

Whenever a person passes in front of the motion sensor:

- The sensor detects movement
- The visitor count increases
- The updated count is displayed on the **LCD screen**

An **LED indicator** is also used to visually indicate detection.

This system is useful in places like:

- Shopping malls
- Offices
- Classrooms
- Smart buildings

It helps in monitoring occupancy and automating entry tracking.

Components Used

- MCU (Microcontroller)
- Motion Sensor
- LCD Display
- LED
- Connecting wires

Step 1:

Open Cisco Packet Tracer and create a new project.

Step 2:

Add the following components:

- MCU (IoT device)
- Motion Sensor

- LCD Display
- LED

Step 3:

Make connections:

- Motion Sensor → Digital Input (Pin 0)
- LCD → Output Pin (Pin 1)
- LED → Output Pin (Pin 2)

Step 4:

Click on MCU → Programming → Select JavaScript

Step 5:

Write and upload the visitor counting program.

Code :

```
var count = 0;

function setup() {
  pinMode(1, OUTPUT);
  pinMode(2, OUTPUT);
  pinMode(0, INPUT);
  customWrite(1, "0");
  Serial.println("Visitor Counting");
}

function loop() {

  if (digitalRead(0) == HIGH) {
    count = count + 1;
    Serial.println("Count: " + count);
    customWrite(1, "TotalEntries:" + String(count));
    digitalWrite(2, HIGH);
    delay(1000);
    digitalWrite(2, LOW);
    delay(2000);
  }
}
```

Step 6:

Start simulation.

Step 7:

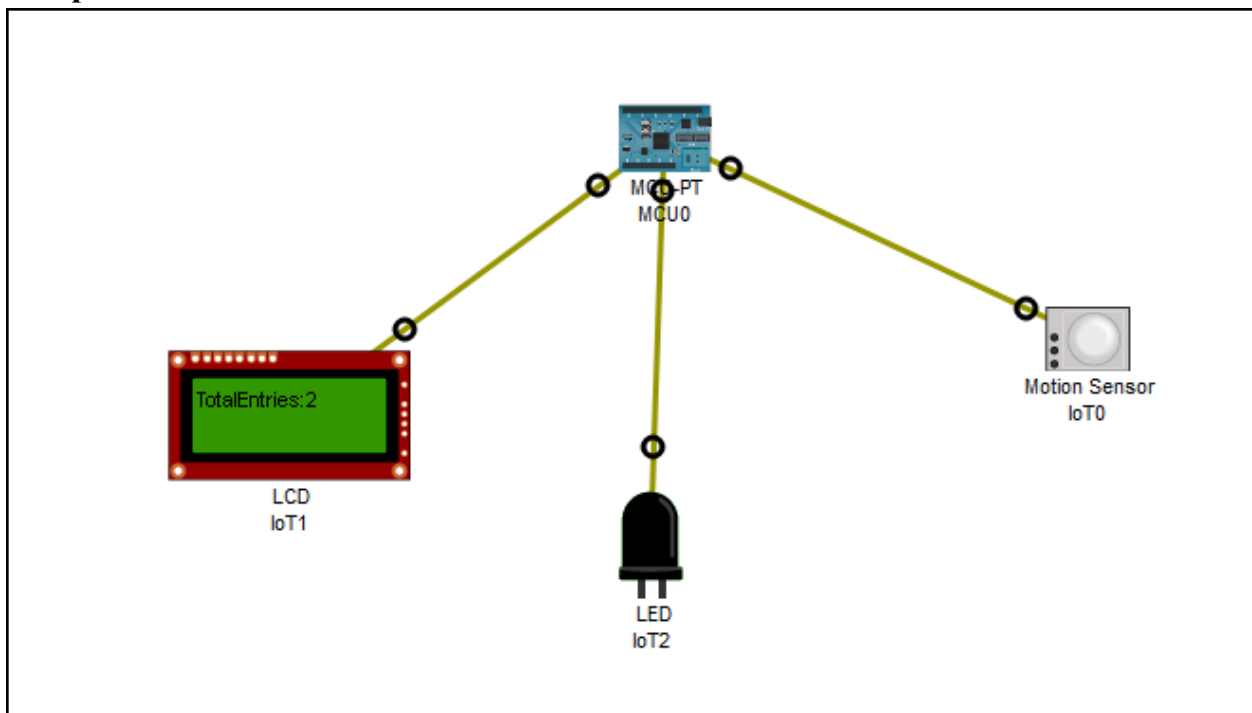
Trigger the motion sensor (simulate movement).

Step 8:

Observe:

- Count increases
- LCD updates with total entries
- LED blinks on detection

Setup :



PRACTICAL NO: 05

Aim: To design and implement an IoT-based carbon monoxide detection system that automatically controls ventilation devices using a sensor in Cisco Packet Tracer.

Description

This project demonstrates a smart safety system where a **carbon monoxide (CO) detector** monitors the level of harmful gas in the environment.

When CO level increases:

- The system **automatically turns ON the fan**
- The **window opens** for ventilation

When CO level is normal:

- The fan turns **OFF**
- The window **closes**

All devices are connected through a **Home Gateway**, enabling remote monitoring using a tablet/smartphone.

This system is useful for:

- Smart homes
- Parking areas
- Industrial safety

Components Used

- Carbon Monoxide Detector
- Fan
- Window
- Home Gateway
- Tablet / Smartphone
- Connecting network (wireless)

Step 1:

Open **Cisco Packet Tracer** and create a new workspace.

Step 2:

Add devices:

- Carbon Monoxide Detector
- Fan
- Window
- Home Gateway
- Tablet

Step 3:

Connect all devices wirelessly to the **Home Gateway**

Step 4:

Register devices:

- Go to **Config** → **IoT Registration**
- Connect each device to Home Gateway

Step 5:

Go to:

👉 **Home Gateway** → **Conditions**

Step 6: Create Rule for HIGH CO Level

- Name: **High CO**

IF:

- Carbon Monoxide Detector → CO Level > 0.15

THEN:

- Fan → Status = **true** (ON)
- Window → Status = **true** (OPEN)

Step 7: Create Rule for NORMAL CO Level

- Name: **Normal CO**

IF:

- Carbon Monoxide Detector → CO Level $\leq$ 0.08

THEN:

- Fan → Status = **false** (OFF)
- Window → Status = **false** (CLOSED)

IoT Monitor					X
IoT Server - Device Conditions			Home Conditions Editor Log Out		
Actions	Enabled	Name	Condition	Actions	
Edit Remove	Yes	SensorHigh	IoT3 Level > 0.15	Set IoT1 Status to High Set IoT0 On to true	
Edit Remove	Yes	SensorLow	IoT3 Level <= 0.08	Set IoT1 Status to Low Set IoT0 On to false	

Step 8:

Enable both rules

Step 9:

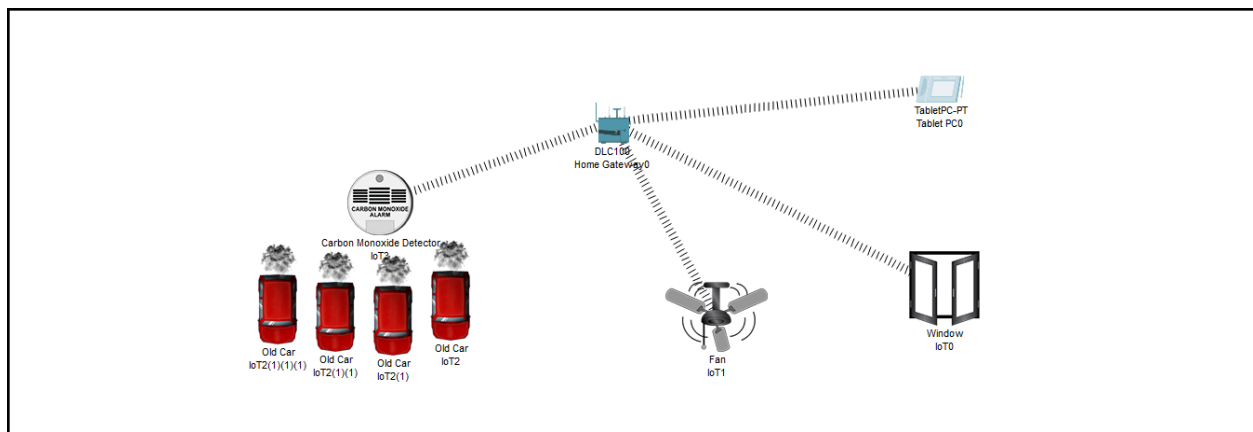
Start simulation

Step 10:

Test:

- Increase CO level → Fan ON + Window OPEN
- Decrease CO level → Fan OFF + Window CLOSE

Setup :

**PRACTICAL NO: 06**

Aim: To design and implement an IoT-based smart security system using a motion sensor that activates an alarm and camera in Cisco Packet Tracer.

Description

This project demonstrates a smart security system where a **motion detector** monitors movement in a specific area.

When motion is detected:

- The **siren (alarm) turns ON**
- The **webcam activates** for monitoring

When no motion is detected:

- The siren turns **OFF**
- The webcam turns **OFF**

All devices are connected through a **Home Gateway**, allowing monitoring via a tablet or smartphone.

This system is useful for:

- Home security
- Office surveillance
- Restricted area monitoring

Components Used

- Motion Detector
- Siren (Alarm)
- Webcam
- Home Gateway
- Tablet / Smartphone
- Wireless network

Step 1:

Open **Cisco Packet Tracer** and create a new workspace.

Step 2:

Add devices:

- Motion Detector

- Siren
- Webcam
- Home Gateway
- Tablet

Step 3:

Connect all devices wirelessly to the **Home Gateway**

Step 4:

Register devices:

- Go to **Config** → **IoT Registration**
- Connect each device to the Home Gateway

Step 5:

Go to:

👉 **Home Gateway** → **Conditions**

Step 6: Create Rule for MOTION DETECTED

- Name: **MotionDetected**

IF:

- Motion Detector → Status = true

THEN:

- Siren → Status = **true** (ON)
- Webcam → Status = **true** (ON)

Step 7: Create Rule for NO MOTION

- Name: **NoMotion**

IF:

- Motion Detector → Status = false

THEN:

- Siren → Status = **false** (OFF)
- Webcam → Status = **false** (OFF)

IoT Monitor					
IoT Server - Device Conditions				Home Conditions Editor Log Out	
Actions		Enabled	Name	Condition	Actions
Edit	Remove	Yes	MotionOn	IoT0 On is true	Set IoT1 On to true Set IoT2 On to true
Edit	Remove	Yes	MotionOff	IoT0 On is false	Set IoT1 On to false Set IoT2 On to false

Step 8:

Enable both rules

Step 9:

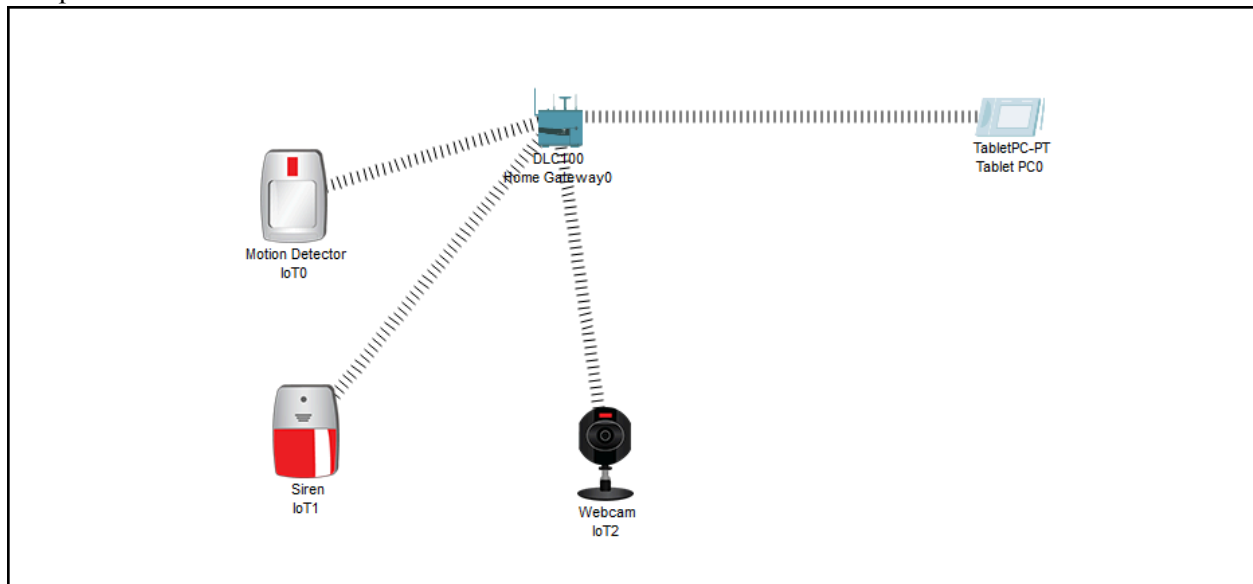
Start simulation

Step 10:

Test:

- Trigger motion → Siren ON + Camera ON
- Stop motion → Siren OFF + Camera OFF

Setup :



PRACTICAL NO: 07

Aim: To design and implement a smart water level monitoring system using IoT in Cisco Packet Tracer that automatically controls a lawn sprinkler based on water level readings.

Description

This project demonstrates an IoT-based automation system where a **water sensor** continuously monitors the water level and sends data to a microcontroller (MCU).

Based on the sensor value:

- If water level is **low**, the system automatically turns **ON the sprinkler** to supply water.
- If water level is **high**, the sprinkler is turned **OFF** to prevent overflow.

An **LED indicator** is also used to show system status.

The system is connected to an **IoT Home Gateway**, allowing monitoring through a smartphone, making it a part of a smart home ecosystem.

This project helps in:

- Water conservation
- Automation of irrigation systems
- Understanding IoT integration

Components Used

- MCU (Microcontroller)
- Water Level Sensor
- LED
- Lawn Sprinkler
- Home Gateway
- Smartphone
- Connecting wires

Step 1:

Open **Cisco Packet Tracer** and create a new workspace.

Step 2:

Add the following devices:

- MCU (from IoT devices)
- Water Level Sensor
- LED
- Lawn Sprinkler
- Home Gateway
- Smartphone

Step 3:

Connect the components:

- Water Sensor → Analog pin (A0) of MCU
- LED → Digital pin (Pin 0)
- Sprinkler → Digital pin (Pin 1)

Step 4:

Connect all IoT devices to the **Home Gateway** wirelessly.

Step 5:

Click on MCU → Programming → Select **JavaScript**

Step 6:

Write and upload the program.

Code:

```
function setup() {  
  pinMode(0, OUTPUT);  
  pinMode(1, OUTPUT);  
  Serial.println("Controlling...");  
}  
  
function loop() {  
  
  var waterLevel = analogRead(A0);  
  Serial.println("Water Level: " + waterLevel);  
  
  var x = map(waterLevel, 0, 1023, 0, 20);  
  
  if (x >= 10) {  
    analogWrite(0, 255);  
    customWrite(1, 0);  
    Serial.println("Water High: " + x);  
  }  
}
```

```
}  
  
else if (x <= 2) {  
    analogWrite(0, 0);  
    customWrite(1, 1);  
    Serial.println("Water Low: " + x);  
}  
  
else {  
    Serial.println("Water Medium: " + x);  
}  
  
delay(1000);  
}
```

Step 7:

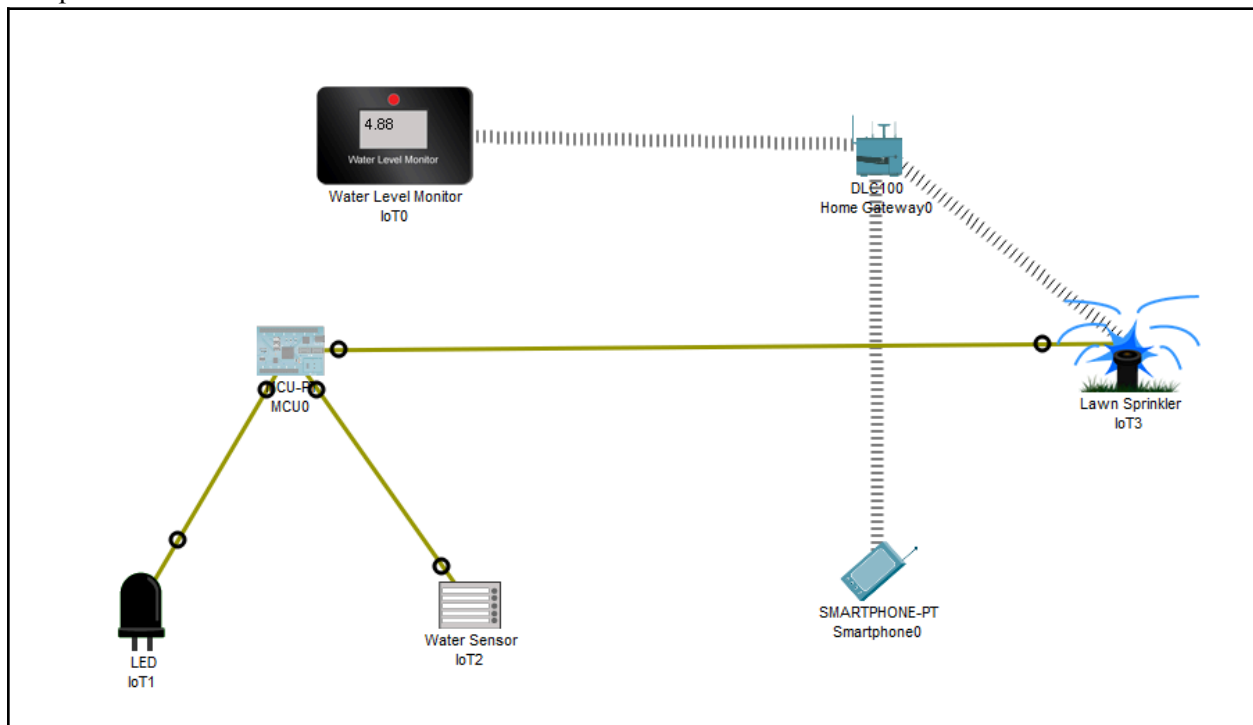
Run the simulation and vary the water level sensor values.

Step 8:

Observe:

- LED turns ON/OFF
- Sprinkler activates/deactivates based on water level

Setup :



PRACTICAL NO: 08

Aim: To design and implement a smart garage door system using a motion sensor and IoT in Cisco Packet Tracer that automatically opens and closes the garage door.

Description

This project demonstrates an IoT-based smart garage system where a **motion sensor** detects the presence of a person or vehicle near the garage.

Based on detection:

- The **garage door opens automatically**
- After a delay, it **closes automatically**

The system is connected to a **Home Gateway**, allowing remote monitoring and control using a smartphone.

This project is useful in:

- Smart homes
- Automated parking systems
- Security-based entry systems

Components Used

- MCU (Microcontroller)
- Motion Sensor
- Garage Door (IoT actuator)
- Home Gateway
- Smartphone
- Connecting wires

Step 1:

Open **Cisco Packet Tracer** and create a new workspace.

Step 2:

Add the following devices:

- MCU
- Motion Sensor

- Garage Door
- Home Gateway
- Smartphone

Step 3:

Make connections:

- Motion Sensor → MCU (Digital Input Pin 0)
- Garage Door → MCU (Output Pin 1)

Step 4:

Connect Home Gateway with:

- Garage Door
- Smartphone (wirelessly)

Step 5:

Go to MCU → Programming → Select **JavaScript**

Step 6:

Write and upload the program.

Code :

```
function setup() {
  pinMode(0, INPUT); // Motion Sensor
  pinMode(1, OUTPUT); // Garage Door

  Serial.println("Smart Garage System Started");
}

function loop() {

  if (digitalRead(0) == HIGH) {
    Serial.println("Motion Detected → Opening Door");
    customWrite(1, 1);
    delay(1000);
    Serial.println("Closing Door");
    customWrite(1, 0);
    delay(1000);
  }
}
```

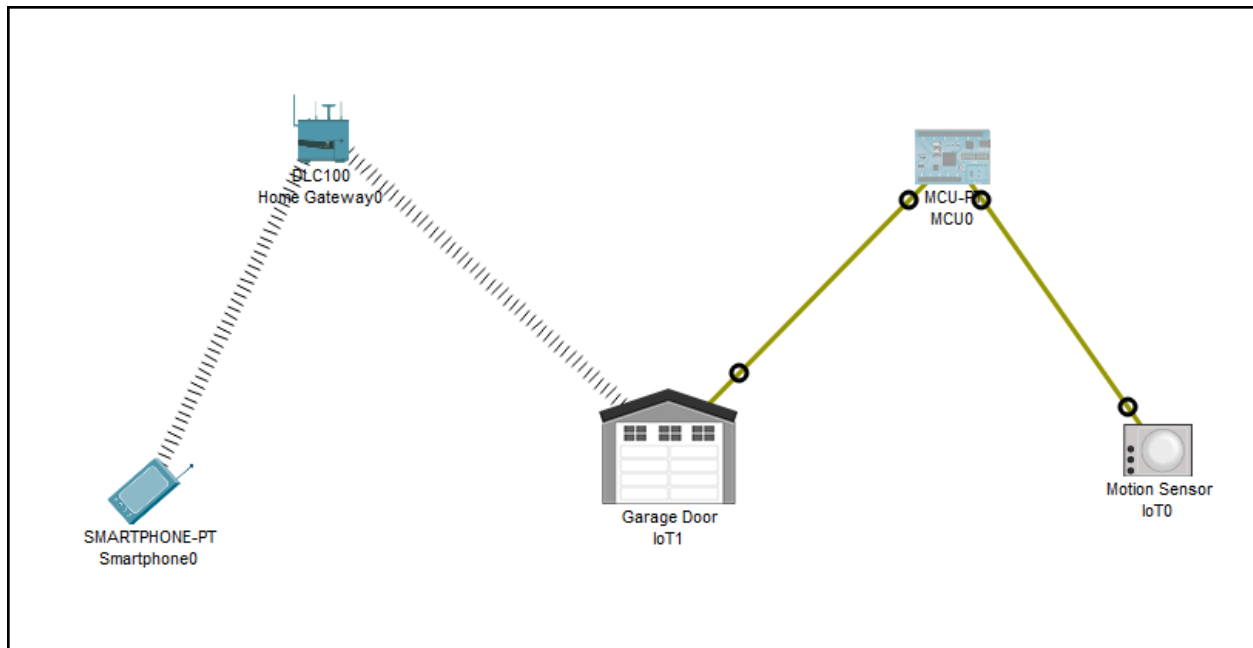
Step 7:

Start simulation.

Step 8:

Trigger motion sensor and observe garage door behavior.

Setup :



PRACTICAL NO: 09

Aim: To develop an IoT-based sprinkler control system using a sensor without using a microcontroller in Cisco Packet Tracer.

Description

In this system, a **water/soil sensor directly communicates with the sprinkler through the IoT Home Gateway**.

Instead of using a microcontroller:

- The system uses **IoT conditions (rules)**
- When sensor detects **low water level (dry condition)** → Sprinkler turns ON
- When water level is **high (wet condition)** → Sprinkler turns OFF

This setup is simpler and uses **built-in IoT automation**.

Components Used

- Water Sensor / Soil Moisture Sensor
- Sprinkler
- Home Gateway
- Smartphone
- Wireless connections

Step 1:

Open **Cisco Packet Tracer**

Step 2:

Add devices:

- Water Sensor
- Sprinkler
- Home Gateway
- Smartphone

Step 3:

Connect all devices to **Home Gateway (wirelessly)**

Step 4: Configure IoT Registration

- Click each device
- Go to **Config** → **IoT Server / Registration**
- Connect them to the Home Gateway

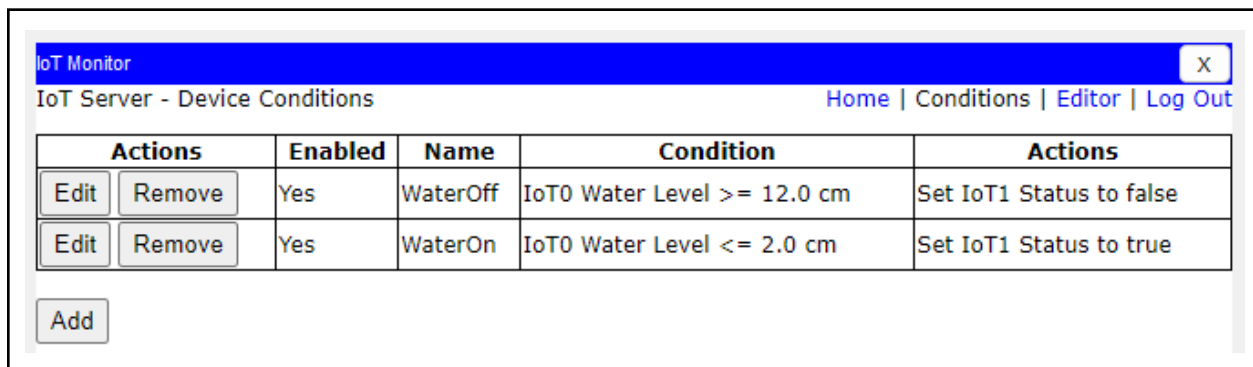
Step 5: Create Automation Rule

Rule 1 (Dry condition)

- **IF** Water Sensor value < threshold
- **THEN** Sprinkler = ON

Rule 2 (Wet condition)

- **IF** Water Sensor value $\geq$ threshold
- **THEN** Sprinkler = OFF



The screenshot shows the 'IoT Monitor' application window. The title bar is blue with 'IoT Monitor' and a close button. Below the title bar, there's a navigation bar with 'IoT Server - Device Conditions' and links for 'Home', 'Conditions', 'Editor', and 'Log Out'. The main content area displays a table of automation rules. The table has columns for 'Actions', 'Enabled', 'Name', 'Condition', and 'Actions'. There are two rules listed: 'WaterOff' and 'WaterOn'. Each rule has 'Edit' and 'Remove' buttons. Below the table is an 'Add' button.

Actions		Enabled	Name	Condition	Actions
Edit	Remove	Yes	WaterOff	IoT0 Water Level $\geq$ 12.0 cm	Set IoT1 Status to false
Edit	Remove	Yes	WaterOn	IoT0 Water Level $\leq$ 2.0 cm	Set IoT1 Status to true

Add

Step 6:

Start simulation

Step 7:

Change sensor value and observe:

- Sprinkler ON (dry)
- Sprinkler OFF (wet)

Setup :

